

1 Introduction

In this project, you will implement three classic AI search and optimization algorithms to solve a grid-based planning problem called **Gold Hunter**. The goal is to find a sequence of actions that collects all gold pieces from a 6×6 grid and returns the agent to the start position, minimizing the total number of actions.

2 Problem Description

You are given a 6×6 grid. Each cell can be:

- Empty: .
- Wall: #
- Gold: G

The agent starts at a designated start cell (S). At each step, the agent may perform one of the following actions:

- Up (U)
- Down (D)
- Left (L)
- Right (R)
- Grab (G)

Rules:

- The agent cannot move into a wall cell.
- The **Grab** action can only be performed when the agent is on a cell that contains gold that has not yet been collected. After grabbing, the gold is collected and the cell becomes empty.
- The goal is to **collect all golds and then return to the start cell**.
- Each action (move or grab) has a cost of 1. The objective is to find a plan (sequence of actions) with the **minimum total number of actions**.

Note: The start cell does not contain gold. Golds may be placed in multiple cells. There is at least one gold in the grid.

3 Input Format

Your program must read the grid description from a **txt** file. The input consists of exactly 6 lines, each containing a string of 6 characters:

- S : start position (exactly one)
- G : gold
- # : wall

- . : empty cell

Example input:

```
S.....
.#.#..
.#G.G.
...#..
..G...
.....
```

4 Output Format

Your program must output **two lines** to standard output:

- Line 1: an integer representing the total number of actions (cost) of the plan.
- Line 2: a string of action letters representing the plan. Actions are: U, D, L, R, G.

If no feasible plan exists, output:

```
-1
None
```

Example output (for the input above):

```
12
R R R D D G L L L U U G
```

5 Tasks

Your tasks include:

1. Implement an A* Search strategy

- Design an **admissible heuristic**.
- Implement the A* algorithm to find an **optimal** plan (minimum number of actions).

2. Select and implement one of the following two algorithms

- Greedy Descent
- Genetic Algorithm

3. Compare A* search with your implemented algorithm

- Compare the running time (in seconds).
- Compare the number of actions taken in the output solutions.
- Discuss the comparison results.

6 Programming Requirements

- Language: **Python**
- Codes must be readable and include comments explaining key steps.
- Write both algorithms in one file and use command-line arguments to choose which to run, e.g.:

```
python goldhunter.py --alg a_star --layout PATH_TO_INPUT_LAYOUT
python goldhunter.py --alg greedy --layout PATH_TO_INPUT_LAYOUT
python goldhunter.py --alg genetic --layout PATH_TO_INPUT_LAYOUT
```

7 Submission

Pack and submit your solution as a **SINGLE ZIP file**, which include:

- A single source code (“`.py`” file)
- A short **report** (PDF) containing:
 - Design explanation for each algorithm
 - For A*: description of the heuristic and a brief proof of admissibility
 - For greedy descent and genetic algorithm: explanation of the representation of candidate solutions and the workflow of the algorithm.
 - Performance analysis (running time, number of actions)

8 Rubrics

- **A* Search** (10 points)
 - Correctly outputs an optimal plan (2 pts)
 - Well-implemented with an admissible heuristic (4 pts)
 - Proof of admissibility (2 pts)
- **Greedy Descent / Genetic Algorithm** (6 points)
 - Design of the solution representation (2 pts)
 - Correct algorithm implementation with explanations (4 pts)
- **Performance Analysis** (4 points)
 - A plot or a table comparing the running time among the three algorithms (1 pt)
 - A plot or a table comparing the total number of actions obtained (1 pt)
 - Reasonable discussion about the results (2 pts)

9 Hints

- You may consider to represent the state as (`position`, `remaining_gold`). Since the grid is small, you can represent the remaining gold by a bitmask.
- For the genetic algorithm, start with a small population and short plan length, then adjust.